

Early-Access Health Scoring: Predicting Post-Launch Retention and Community Climate from Public Steam Data

Configuration Manual

Saurabh Ganesh Mastud

1 Introduction and scope

This manual sets out how to reproduce the Early-Access Health Scoring project from this repository: the machine it was built on, what to install, the order the notebooks run in, and how to view the results. It assumes no prior Python experience.

The report in `paper/` is a separate document and carries the findings. If a number appears in both, the report is the authority.

The pipeline collects public Steam review histories for early-access games, scores the review text for toxicity and sentiment, reduces everything to one row per game, and fits models that anticipate post-launch retention and community climate. The output is a health score from 0 to 100 per game, a risk band, and a static website presenting both.

You need a Windows machine, a local clone of this repository, and an internet connection for the install. Anything you type appears in a box like this:

```
python --version
```

Each step states what a success looks like, so a silent failure is visible before you move on.

2 Hardware and software environment

Table 1 gives the machine the reported results were produced on. Only the graphics card really matters: toxicity scoring runs on it, and on a processor alone that stage takes hours instead of minutes. Memory is not a constraint because no stage loads the whole corpus at once. Windows is not required either, though the paths here use backslashes.

Four pieces of software are needed: **Python 3.11.9**, **Visual Studio Code** for the notebooks, **Node.js 22.20.0** for the website, and the **CUDA build of PyTorch 2.11.0+cu128**, which installs separately from the rest.

The Python version is not a suggestion. It must be 3.11. Version 3.13 is the current default on most machines and PyTorch publishes no CUDA build for it, so the toxicity stage fails there with a missing-module error that does not point at the version.

Table 2 lists every package the project imports directly. These are pinned in `notebooks\requirements.txt`, so they install with one command.

Table 1: The development machine.

Component	Specification
Operating system	Windows 11 Home Single Language, build 26200, 64-bit
Processor	Intel Core i7-14700HX, 20 cores, 28 threads
Memory	15.7 GB RAM
Graphics	NVIDIA GeForce RTX 4050 Laptop GPU, 6.4 GB VRAM
Free disk space	about 8 GB for a full run, including the raw review layer

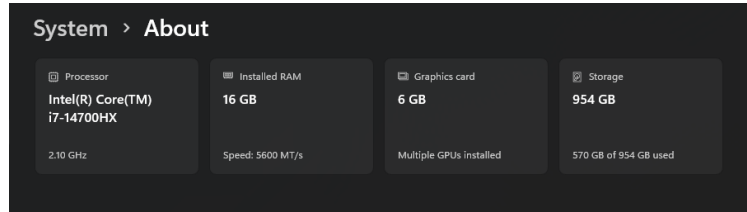


Figure 1: Windows system information for the development machine.

3 Installing the environment

Six steps, in order. Each ends with a check; do not move on until it passes. Step 6 is only needed if you want the website as well as the analysis.

1. Install Python 3.11

Take the Windows 64-bit installer for **3.11.9** from <https://www.python.org/downloads/release/python-3119/> and tick **Add python.exe to PATH** on the first screen. If a newer Python is already present, install 3.11 alongside it; the `py -3.11` launcher below picks the right one.

Table 2: Direct dependencies, read from the working environment on 6 August 2026.

Package	Version	Package	Version	Purpose
pandas	2.3.3	numpy	2.4.6	tables and maths
pyarrow	24.0.0	PyYAML	6.0.3	files and settings
requests	2.34.2			collection
torch	2.11.0+cu128	transformers	5.12.1	the toxicity model
detoxify	0.5.2	vaderSentiment	3.3.2	text scoring
scikit-learn	1.9.0	scipy	1.17.1	models and tests
shap	0.51.0	mlflow	3.15.0	attribution, tracking
matplotlib	3.11.0	streamlit	1.60.0	figures, demo app
ipykernel	7.3.0			notebook runtime

```
py -3.11 --version
```

Check: prints Python 3.11.9. If py is not recognised, the PATH box was missed; re-run the installer and choose Modify.

2. Create the environment

```
cd D:\ea-health-scoring
py -3.11 -m venv .venv
.venv\Scripts\activate
```

Check: the prompt gains a (.venv) prefix. It must be activated in every new terminal.

3. Install PyTorch with CUDA

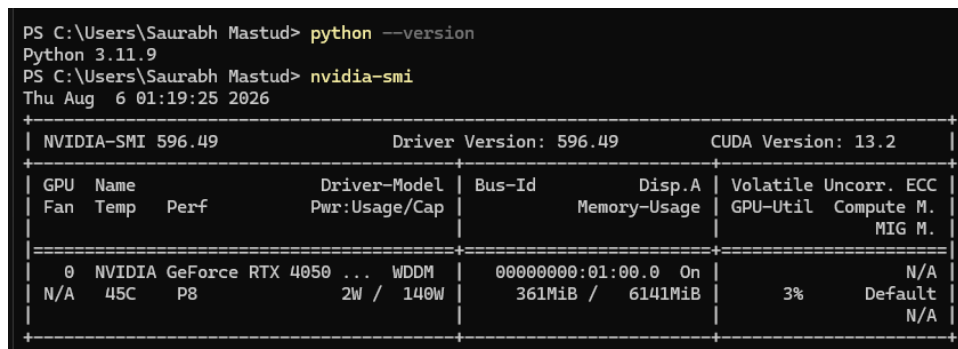
PyTorch comes first and from its own index, because the CUDA build is not on the ordinary one.

```
pip install torch==2.11.0 --index-url https://download.pytorch.org/whl/cu128
python -c "import torch; print(torch.__version__, torch.cuda.is_available())"
```

Check: prints 2.11.0+cu128 True. Around 2.5 GB downloads. With no NVIDIA card, use `pip install torch==2.11.0` instead; everything still runs, but scoring takes hours rather than minutes.

4. Install everything else

```
pip install -r notebooks\requirements.txt
```



The image shows a terminal window with the following content:

```
PS C:\Users\Saurabh Mastud> python --version
Python 3.11.9
PS C:\Users\Saurabh Mastud> nvidia-smi
Thu Aug  6 01:19:25 2026
```

NVIDIA-SMI 596.49		Driver Version: 596.49			CUDA Version: 13.2		
GPU	Name	Driver-Model	Bus-Id	Disp.A	Volatile	Uncorr.	ECC
Fan	Temp	Pwr:Usage/Cap		Memory-Usage	GPU-Util	Compute M.	MIG M.
0	NVIDIA GeForce RTX 4050	WDDM	00000000:01:00:0	On			N/A
N/A	45C P8	2W / 140W	361MiB / 6141MiB		3%	Default	N/A

Figure 2: Confirming the interpreter and the graphics card. The CUDA version `nvidia-smi` reports is the highest the driver supports, not the one PyTorch uses; a 13.2 driver runs the 12.8 build without complaint, so the two differing is expected.

5. Register the notebook kernel

The notebooks are pinned to a named kernel so they cannot run under the wrong Python by accident.

```
python -m ipykernel install --user --name ea-health-py311 --display-name "Python 3.11 (EA Health)"
```

Install Visual Studio Code from <https://code.visualstudio.com/>, add its **Python** and **Jupyter** extensions, then open the `final_thesis` folder.

Check: open any notebook, click the kernel name at the top right, and confirm **Python 3.11 (EA Health)** is selected.

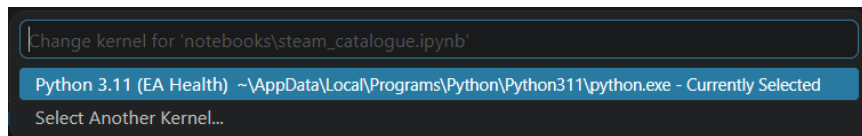


Figure 3: The kernel picker. The entries named `python3` or `python3.13` point at a different interpreter with no GPU build of PyTorch, and stage 5 fails under them.

6. Install Node.js, for the website only

Node.js 22 or newer from <https://nodejs.org/>, LTS installer, defaults accepted.

```
cd web\website
npm install
```

Check: `node --version` prints `v22.20.0` or higher and `npm install` ends without an error block.

4 The project folder and its configuration

Nothing needs moving or renaming: every path in the code is relative to the `final_thesis` folder. `notebooks\` holds the eight pipeline notebooks, `config.yaml` and `requirements.txt`. `data\raw\` holds the collected review histories and a checkpoint per game; `data\processed\` holds everything derived from them, and `data\ml_runs\` the MLflow database. `web\` holds the Astro site, the Streamlit application and the script that builds the data bundle both read. `figures\` and `reports\` hold the charts and the two documents.

Every path, threshold, band cutoff and chart colour lives in `notebooks\config.yaml`, which the notebooks, the website and the demo application all read. A value written anywhere else would be a defect, and that is why the report, the site and the model cannot disagree about a cutoff. Table 3 covers the blocks worth knowing before you run anything.

One thing not to change. `data\raw\` is written once and never edited. Every correction happens downstream of it, so any result traces back to the bytes that came off the endpoint.

Table 3: The settings that matter in `config.yaml`.

Block	What it controls
seed	fixed at 42, used by every random step so a re-run reproduces the same numbers
collection_date	2026-08-01, the day playtime was snapshotted. Changing it re-collects nothing
paths	where each stage reads and writes
frame	which games are modelled: at least 200 English early-access reviews and 50 after launch, giving 545
split	temporal, on launch date, 70 per cent training
bands	the At Risk and Healthy cutoffs, 43.6 and 57.2, written back by stage 7 from the training data. Never type these by hand
predictors	the explicit list of columns the model may use
palette	every colour used in the charts, the site and the application

5 Running the pipeline

Eight notebooks run in the order in Table 4, which is not the alphabetical order the folder shows. Each stage reads the previous stage’s files from disk and writes its own; nothing is passed in memory, which is what makes the sequence safe to stop and restart.

Table 4: The eight stages, in run order, with what to expect on time.

#	Notebook	Writes	From nothing	On this folder
1	steam_catalogue	the screened candidate list	45 min	not needed
2	review_collection	a review file and checkpoint per game	2 h 15 m	skipped
3	auxiliary_sources	per-game metadata	a few min	under a min
4	review_cleaning	cleaned English reviews	30 min	under a min
5	toxicity_scoring	toxicity and sentiment scores	1 to 2 h on GPU	under a min
6	feature_engineering	one row per game	10 min	10 min
7	health_score_model	scores, bands, result tables	2 to 3 min	2 to 3 min
8	figures	the eleven charts	1 min	1 min

Stages 1 to 3 do not need running. Their output ships in the folder and they account for almost all the time. Starting at stage 4 reproduces every number in the research report. Run them only to collect a fresh corpus, in which case the results will differ, because Steam has moved on since 1 August 2026.

To run a stage: open the notebook, confirm the kernel reads **Python 3.11 (EA Health)**, then choose **Restart Kernel and Run All Cells**. Do not run cells individually and out of order, which is the one habit that produces plausible wrong numbers with no error. Every notebook ends with an assertion cell checking its own output; if that cell prints its summary without raising, the stage finished.

Stage 1: the candidate list

Enumerates every multiplayer game on the store and screens each for an early-access history and enough reviews to measure. Of 21,330 titles it keeps 1,887.

Stage 2: collecting the reviews

The long one: 628,389 requests over about two and a quarter hours with eight workers. It is rate-limited and backs off when the endpoint asks, so do not raise the worker count. It is also resumable, because the checkpoint is written before the review file is renamed into place. The shipped corpus was in fact collected across an interruption.

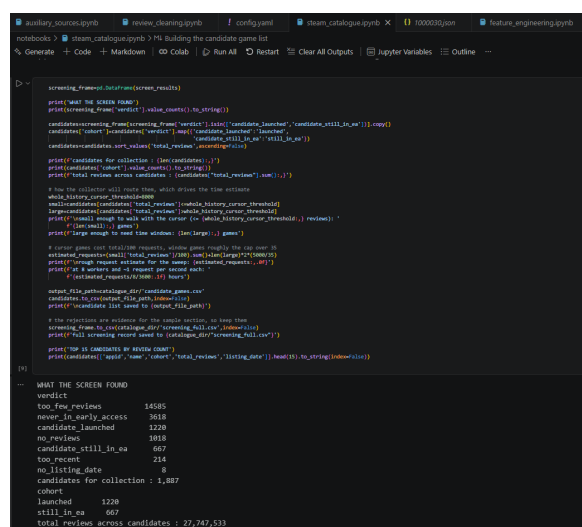


Figure 5: The stage 1 screening result. Most of the catalogue is rejected for too few reviews.

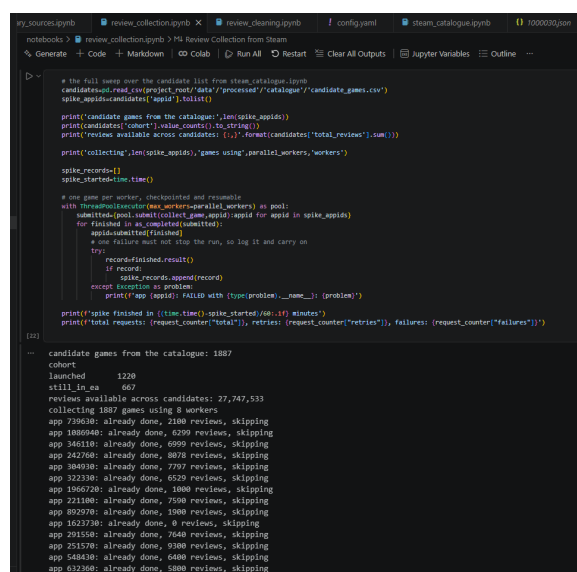


Figure 6: Collection in progress, one game per line. A game already on disk is skipped.

Trap. Steam’s own `app_release_date` is not the date a game left early access; it differs by over a week for 74.8 per cent of launched titles. Stage 2 derives the real transition by binary search over the review stream, and every later stage uses that derived date.

Stages 3 and 4: metadata and cleaning

Stage 3 fetches app details, skipping anything already on disk. Stage 4 strips markup and links, drops non-English reviews, hashes the author identifier, and keeps Steam’s profanity mask as a countable token rather than deleting it. From 3,504,533 collected reviews it keeps 1,865,234 English ones. Games already cleaned are skipped.

Stage 5: toxicity and sentiment scoring

The stage that needs the graphics card. Detoxify scores toxicity, VADER scores sentiment, and 1,512,547 rows are written. The first run downloads the model weights once. Scoring is capped at 2,000 reviews per game, spread across eight time periods within each

```

1 {
2   "appId": "1000000000",
3   "release_timestamp": "1580324436",
4   "release_date": "2020-01-29",
5   "total_reviews": 2382,
6   "total_positive": 2083,
7   "total_negative": 219,
8   "review_score_desc": "Very Positive",
9   "still_in_early_access": false,
10  "listing_timestamp": "1580324436",
11  "cohort": "launched",
12  "derived_launch_timestamp": "1602197025",
13  "launch_date": "2020-10-06",
14  "listing_vs_launch_delta_days": 253.2,
15  "bisect_probes": 14,
16  "bisect_bracket_hours": 15.6,
17  "window_method": "n/a",
18  "collection_strategy": "whole_history_cursor",
19  "done": true,
20  "n_reviews": 2382,
21  "n_flagged_early_access": 760,
22  "n_es_window": 2382,
23  "n_post_window": 0,
24  "truncated_window": 0,
25  "cap_was_hit": false,
26  "window_abandoned_at_cap": 0,
27  "requests_used": 316,
28  "seconds_taken": 29.5,
29  "collected_at": "2020-08-01T02:17:22",
30  "corpus_manifest_id": "corpus-20200801-ruhealth-v1",
31  "collection_run_id": "20200801T044257"
32 }

```

Figure 7: One checkpoint. Besides the counts it records the derived launch date and the binary-search probes the later stages read.

```

111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000

```

Figure 8: Cleaning, one game per line. Dropped rows are counted, never silently discarded.

of the early-access and post-launch phases so a quiet stretch keeps its representation; the cap binds on 318 of 1,793 games.

The first cell of this stage prints whether the card is visible. `cuda available : False` still runs, on the processor, in hours rather than minutes. The resulting score distribution skews hard to zero, as a healthy result should: about three per cent of reviews score above 0.5.

Stages 6 to 8: features, models and figures

Stage 6 reduces the scored reviews to one row per game, 1,793 rows and 35 columns, and computes the retention measure as the gap between a reviewer's lifetime playtime and their playtime when writing. Stage 7 narrows to the 545 games with enough reviews either side of launch, splits them by launch date into 381 training and 164 test, fits the models and writes the nineteen result tables. Stage 8 draws the eleven charts.

Everything fitted, including the scaling and the band cutoffs, is fitted on the training partition only, and the test partition is scored once at the end.

6 Data artefacts

Table 5 lists what the pipeline leaves on disk. A full run comes to about 8 GB, almost all of it the raw review layer, which is not redistributed in this repository.

The two tables worth opening. `game_features.parquet` has one row per collected game and holds the measures the model may see. `game_scores.parquet` has one row per scored game, 545 modelled plus 424 held out for the survivorship comparison, and holds the score under both weightings, the two components, the band and the per-game attributions. It is the file the website reads. Both are parquet rather than CSV so column types travel with the data:

```
python -c "import pandas as pd; print(pd.read_parquet('data/processed/game_scores.parquet'))"
```


The result tables. The nineteen CSV files in `data\processed\tables` are one per reported analysis, and every number in the research report traces to one of them. Three data dictionaries alongside them document every column in both parquet files.

What is not redistributed. Collected review text stays in `data\raw` and is not published. Steam’s terms permit collecting review data, not republishing the corpus, so any public release carries derived game-level features and the collection code only. Author identifiers are hashed at the cleaning stage, which is pseudonymisation rather than anonymisation.

7 Model training and experiment tracking

Stage 7 does all the training. Table 6 records the settings, all read from `config.yaml` rather than typed into the notebook.

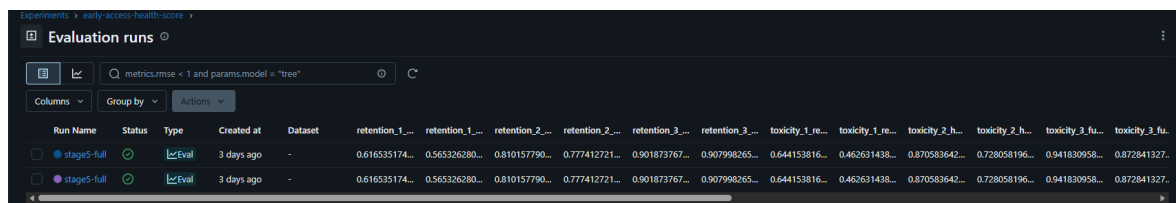
Table 6: Training configuration.

Setting	Value
Modelling frame	545 games, at least 200 English early-access reviews and 50 post-launch
Held-out cohort	424 games still in early access, never used for training
Split	temporal on launch date: 381 train, 164 test
Random seed	42
Predictors	19, an explicit list rather than every column minus the outcome
Significance level	0.01667, Bonferroni-corrected over three hypotheses
Band cutoffs	43.6 and 57.2, from training terciles, written back to <code>config.yaml</code>

Every fitted run is logged to MLflow. Version 3 dropped the plain file store, so the record is a SQLite database rather than a folder of files, which changes only the command used to open it:

```
mlflow ui --backend-store-uri sqlite:///data/ml_runs/mlflow.db
```

Then open `http://127.0.0.1:5000`; Ctrl+C stops it. Note the forward slashes. The `sqlite:///` prefix is a URI, not a Windows path, and backslashes are rejected there even on Windows.



Run Name	Status	Type	Created at	Dataset	retention_1...	retention_1...	retention_2...	retention_2...	retention_3...	retention_3...	toxicity_1_re...	toxicity_1_re...	toxicity_2_h...	toxicity_2_h...	toxicity_3_fu...	toxicity_3_fu...
stages5-full	Completed	Eval	3 days ago	-	0.616535174...	0.565326280...	0.810157790...	0.777412721...	0.901873767...	0.907998265...	0.644153816...	0.462631438...	0.870583642...	0.728050196...	0.941830958...	0.872841327...
stages5-full	Completed	Eval	3 days ago	-	0.616535174...	0.565326280...	0.810157790...	0.777412721...	0.901873767...	0.907998265...	0.644153816...	0.462631438...	0.870583642...	0.728050196...	0.941830958...	0.872841327...

Figure 13: The run list. Each row is one fitted model with the metrics it scored.

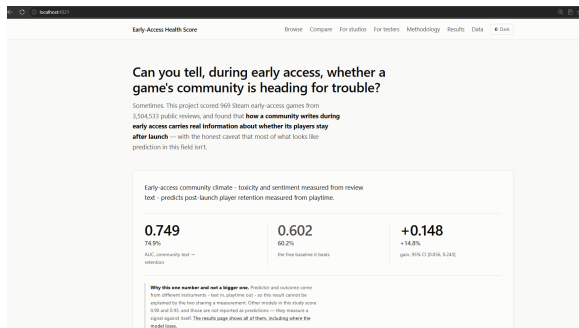


Figure 15: The site as it opens. The corpus size and collection date are stated on the page rather than left implicit.

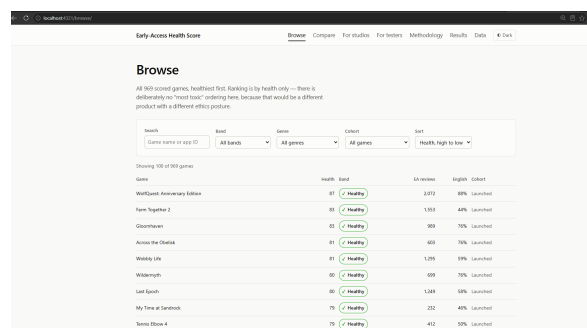


Figure 16: The browse view. Per-game pages reachable from here show the two components, the attribution, and the limitations applying to that title.

Early-Access Health Score

969 Steam games scored from 3,504,533 public early-access reviews. Corpus collected 2026-08-01.

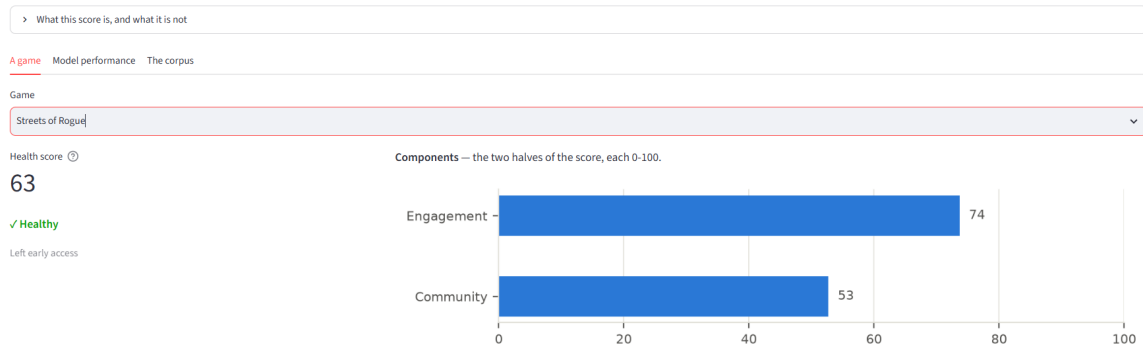


Figure 17: The demo application, reading the same bundle as the static site.